

Kiss ISA Reference Manual

License Notice Copyright © 2026 Wolven LLC. This document is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). See [docs/Licenses/CC-BY-4.0.txt](#) and <https://creativecommons.org/licenses/by/4.0/>.

Version 1.0 Draft

1. Introduction

1.1 Purpose

This manual presents the Kiss Instruction Set Architecture. It combines the core architectural context of the Kiss processor with a command-oriented description of the ISA command families, their command-bit usage, and the operand conventions needed to read and write Kiss instructions.

1.2 Scope

This manual covers the Kiss ISA command set and the architectural rules needed to interpret it. It focuses on the core command families and the conventions shared across them.

1.3 Related Kiss documents

This manual should be read alongside the command inventory, assembler command list, assembler grammar, and the broader Kiss ISA reference notes.

2. ISA Design Goals

The Kiss instruction set architecture is designed around six principles that distinguish it from conventional ISAs.

2.1 Fixed-Width 100-Bit Instructions

Every Kiss instruction is exactly 100 bits wide. There are no variable-length encodings, instruction prefixes, or multi-byte opcode escapes.

This removes the pre-decode and instruction-boundary complexity common in variable-length ISAs. The decoder always knows where each instruction begins and ends.

The 100-bit width is the natural unit size of the architecture: a 10-bit byte times 10 bytes equals

100 bits.

2.2 One-Based Indexing

All numbering and indexing in Kiss starts at 1, not 0.

Hardware: instruction bits are numbered 1-100, general-purpose banks are R1-R5, thread IDs are 1-5, and stack IDs are 1-10.

Software: array indices, string positions, loop counters, and numbered resources also begin at 1.

Rationale: zero is consistently available as an invalid or unset value, reducing off-by-one confusion.

2.3 Three Data Types, Three ALUs

Kiss defines three fundamental data types.

Type	Representation	ALU	Primary Operation
Binary	Straight binary, unsigned	Adder	Addition
Integer	Ones complement, signed	Subtractor	Subtraction
Decimal	Packed 5-bit digits, ones complement	Matrix lookup tables	Table-driven digit arithmetic

The sign convention for signed types is consistent: bit 1 means positive and bit 0 means negative.

2.4 In-Order Execution with SMT Latency Hiding

Kiss uses in-order execution. Instead of out-of-order speculation, memory latency is hidden through hardware multithreading. Each core supports five hardware threads, allowing the core to switch to another ready thread when one stalls.

2.5 Register-Register Architecture

Kiss is a load-store architecture. Arithmetic and logic operations act on registers, while memory is accessed through explicit Load and Store instructions.

2.6 10-Bit Bytes

A Kiss byte is 10 bits.

- **Packed decimal efficiency:** two 5-bit decimal digits fit exactly in one byte.
- **Wider character space:** one byte can represent 1,024 code points.
- **Natural base-10 scaling:** 10-bit bytes, 100-bit instructions, and larger powers-of-10 structures align cleanly.

3. Register Architecture

3.1 The Register Model

Every Kiss register bank is a single flat bit array. What programmers see as “registers” — **R1**, **R1U**, **R1WA**, **PC**, **SF**, and so on — are not physically separate storage locations. They are **named windows** onto the bit array, each defined by a starting bit position and a length.

A Register ID is essentially a triple:

Field	Meaning
Bank	Which physical bit array (e.g., R1 bank, R7 bank, special-register bank)
Starting position	1-based bit index where the window begins within the bank
Length	Number of bits the window covers

Writes through any Register ID propagate to every overlapping Register ID. For example, **R1WA** covers bits 1–100 of the R1 bank; **R1W** covers bits 1–500; **R1U** covers bits 1–1000; **R1** covers all 2000 bits. Writing to **R1WA** modifies the first 100 bits of all four windows simultaneously, because they share the same physical storage.

This model has two practical implications:

- Programmers select the Register ID whose size matches the data they are working with — a 100-bit window for a Binary or Integer value, a 200-bit window for a maximum-size Decimal value, a 50-bit window for an address. The window size determines what the ALU operates on; the underlying bits remain a single contiguous array.
- Aliasing is intentional, not a hazard. The compiler and assembler use overlay relationships explicitly when sub-fields of a wide value need to be addressed individually.

The complete enumeration of Register IDs, their banks, starting positions, and lengths is generated from the **KSReg** database table and lives in [docs/kiss-registers.md](#).

3.2 General-Purpose Register Banks

Five general-purpose register banks per hardware thread: **R1 through R5**. Each bank is 2000 bits.

Every GP bank uses the same hierarchical set of overlay windows:

Window Level	Size	Count per Bank	Example IDs (R1 bank)
Full bank	2000 bits	1	R1
Half bank	1000 bits	2	R1U, R1V
Quarter bank	500 bits	4	R1W, R1X, R1Y, R1Z
Typed-value window	200 bits	many	maximum-size Decimal fits exactly

Window Level	Size	Count per Bank	Example IDs (R1 bank)
100-bit views	100 bits	many	R1WA-R1WE, R1XA-R1XE, ...
Smaller	every byte-aligned size from 10 to 190 bits, plus 25 bits	many	fine-grained access into the wider views; no 5-bit GP registers (5-bit slots exist only in the special bank)

The 200-bit window is the natural size for typed-value operations — every Kiss value, regardless of type, fits in one 200-bit register: a maximum 40-digit Decimal value (20 packed-digit bytes = 200 bits, or 39 digits + 1 DecPt-marker slot for fractional Decimals), or a maximum 100-bit Binary/Integer value with room to spare. After the 2026-05-14 KSReg restructure there is a dedicated register window for every Decimal byte size from 1 to 20 (10-bit through 200-bit, in 10-bit steps), so no Decimal value is forced into a wider window with wasted bits. The 100-bit window remains the natural size for a maximum Binary or Integer value.

3.3 Auxiliary Register Banks

Bank	Size	Purpose
R6	—	Reserved for future use
R7	1000 bits	Bit-manipulation workspace — dedicated to bit-field operations
R8	1000 bits	Compiler scratch — used for materializing immediates and intermediate values

R7 and R8 use the same overlay-window scheme as the upper half of a GP bank (1000 bits with 500/100/50/etc. subdivisions), but they are not part of any GP bank.

3.4 Status Flags

Status flags live in a single Status Flag (**SF**) register — 30 bits wide (3 Kiss bytes) — with each flag individually addressable by name. The “Not” variants (**NZ**, **NE**, **NCry**, etc.) are simply the corresponding flag bit being 0; no extra hardware is needed.

Flags are organized into three independent groups (bits assigned MSB-first per Kiss convention):

Group	Bits	Set by	Cleared by
Arithmetic	1-7, 22-23	Arithmetic / Shift / Cvt operations with Set modifier	Same ops on next instance
Result-property	8-12	Arithmetic ops + Test	Same ops on next instance
Compare	13-21	Compare and ChkBnds	Next Compare / ChkBnds
Reserved	24-30	—	—

The Arithmetic group spans two non-contiguous bit ranges — 1-7 hold the primary arithmetic signals, 22-23 hold the truncation-direction signals (THi/TLo) added in 2026-05-16 after bits 8-21 were already assigned. The split is purely a result of insertion order; both ranges are written by the same op classes.

Arithmetic Flags — bits 1-7 (primary)

Set by arithmetic and logical operations:

Bit	Flag	Meaning	Determination
1	OVR	Mathematical result exceeded destination's positive range	Set when the <i>mathematical</i> result of an operation lies above the destination's representable maximum. Add/Sub modular wraps from the positive side, Mul/Div precision overflows on the positive side, Div by zero with positive-signed quotient. Pure directional — does NOT imply which side got truncated (that's THi/TLo).
2	UND	Mathematical result exceeded destination's negative range	Set when the mathematical result lies below the destination's representable minimum. Add/Sub modular wraps from the negative side, Mul/Div precision overflows on the negative side, Div by zero with negative-signed quotient. Pure directional — does NOT cover Cvt narrowing or Fixed-rescale truncation (those use THi/TLo).
3	WRP	Modular wrap (Add/Sub only)	Set ONLY when an Add or Sub produced an OVR or UND AND that out-of-range outcome was a modular wrap. Strict sub-case of OVR/UND for Add/Sub: never fires alone, never fires for Mul/Div/Shift/Cvt.
4	CRY	Add result didn't fit	Set after Add when result exceeded representable range in either direction (OVR v UND). Direction-agnostic — internal end-around-carry behavior during the addition itself is not what CRY measures. Also serves as the chain bit in WithCarry mode. Add never sets BRW.
5	BRW	Sub result didn't fit	Set after Sub when result exceeded representable range in either direction (OVR v UND). Direction-agnostic — internal bit-level borrows that don't cause final out-of-range (e.g., $+50 - +60 = -10$) do <i>not</i> fire BRW. Also serves as the chain bit in WithBorrow mode. Sub never sets CRY.
6	INF	Result is $\pm$ infinity	Set after Dec operations whose result is $\pm\infty$. Bin/Int never set INF.
7	ANY_CARRY	Decimal carry/borrow digit pending	Set when Dec arithmetic produces a non-null overflow carry digit available in SCRY/SBRW

Arithmetic Flags — bits 22-23 (truncation-direction)

Set by arithmetic / shift / Cvt operations alongside the bits 1-7 flags. THi/TLo describe *which end* of the digit/bit array was truncated; they are independent of OVR/UND (which describe mathematical direction).

Bit	Flag	Meaning	Determination
22	THi	High-end digits/bits were dropped	Set when an operation discarded data at the most-significant end. Fires alongside OVR/UND for Mul/Div precision overflows. Fires alone for Shift-left bit loss and Cvt narrowing (no directional component for those operations). Add/Sub never set THi — their truncation is signaled by WRP.
23	TLo	Low-end digits/bits were dropped	Set when an operation discarded data at the least-significant end. Fires alone for Fixed-rescale truncation, Div nonzero-remainder, Shift-right bit loss, and Floating-Dec precision underflow. Pure precision-loss signal — no directional component, no OVR/UND ever co-fires.

Combination cases:

Outcome	OVR	UND	WRP	CRY	BRW	INF	THi	TLo
Add wrap positive (pos+pos overflow)	✓		✓	✓				
Add wrap negative (Int/Dec neg+neg overflow)		✓	✓	✓				
Sub wrap negative (below min)		✓	✓		✓			
Sub wrap positive (Int/Dec opposite-sign overflow)	✓		✓		✓			
Mul precision overflow positive	✓						✓	
Mul precision overflow negative (Int/Dec only)		✓					✓	
Div precision overflow positive	✓						✓	
Div precision overflow negative (Int/Dec only)		✓					✓	
Div with nonzero remainder								✓
Div by zero, positive-signed quotient (Bin/Int/Dec)	✓					(Dec) ✓		
Div by zero, negative-signed quotient (Int/Dec)		✓				(Dec) ✓		
Div 0 / 0 (any type)								
Fixed-rescale of low fractional digits								✓
Floating-Dec precision underflow								✓
Shift left bit loss							✓	
Shift right bit loss								✓
Cvt narrowing high-bit loss							✓	

Div 0/0 = 0 with no flag follows the Kiss “no NaN” rule (see §“Infinity Arithmetic” in the ISA reference). Div-by-zero direction follows **dividend_sign XNOR divisor_sign** — Kiss has signed zeros (+0, −0) so divisor sign matters. **Cvt narrowing fires THi alone — never OVR — even when the source value can’t fit in the destination’s range.** Cvt is conceptually a data-movement operation, not arithmetic with directional overflow.

Result-property Flags — bits 8-12

Set by arithmetic ops (alongside the Arithmetic group) and by **Test** (alongside the Compare group):

Bit	Flag	Meaning	Determination
8	Z	Result is zero	wide NOR of all result bits
9	Pos	Result is positive	sign bit = 1
10	Neg	Result is negative	sign bit = 0

Bit	Flag	Meaning	Determination
11	EVN	Result is even	low bit is 0 (Binary/Integer); least significant non-null digit is 0/2/4/6/8 (Decimal)
12	ODD	Result is odd	low bit is 1 (Binary/Integer); least significant non-null digit is 1/3/5/7/9 (Decimal)

Compare Flags — bits 13-21

Independent of the arithmetic and result-property groups — an arithmetic op does not clobber compare results. Splits into a shared equality flag plus two orthogonal directional quartets, each set by a distinct ISA command:

Bit	Flag	Meaning	Determination	Set by
13	EQ	Most recent compare found equality	order-equal (Compare/ChkBnds) OR magnitude-equal (CmpMag)	Compare, ChkBnds, CmpMag
14	GT	First > second	subtraction result positive	Compare, ChkBnds
15	GE	First ≥ second	GT OR EQ	Compare, ChkBnds
16	LT	First < second	subtraction result negative	Compare, ChkBnds
17	LE	First ≤ second	LT OR EQ	Compare, ChkBnds
18	Abv	first > second	magnitude of first exceeds magnitude of second	CmpMag (<i>future</i>)
19	AbvE	first ≥ second	Abv OR EQ	CmpMag (<i>future</i>)
20	Blw	first < second	magnitude of first below magnitude of second	CmpMag (<i>future</i>)
21	BlwE	first ≤ second	Blw OR EQ	CmpMag (<i>future</i>)

Compare is Kiss’s universal natural-ordering compare. Kiss’s data type encodings are co-designed so raw register-level ordering coincides with semantic ordering for every type — Bin (trivially unsigned), Int (ones-complement reversed-sign places all negatives below all positives in raw bits), Decimal (signed-digit codes ordered to match decimal ordering), and Text (codepoint = byte order). One raw compare therefore produces correct ordering for any datatype; the CMB 1-2 type field on **Compare** is informational, not behavior-changing. No mainstream ISA has this property — x86, ARM, RISC-V, and MMIX all need separate signed/unsigned compares because two’s-complement breaks ordering coherence.

CmpMag is genuinely orthogonal: it asks **|a|** vs **|b|**, ignoring sign — which diverges from natural ordering for signed types when operands cross zero. Useful for iterative-solver convergence checks (**|residual| < tolerance**), AI-core activation thresholds, and magnitude-sort ordering.

CmpMag is **spec-defined but not yet implemented**; flag bits 18-21 are reserved for it and no current Kiss instruction writes them. EQ is shared: **CmpMag** writes it on magnitude-equal.

Total: 23 active flags fit in the 30-bit SF register (3 Kiss bytes), with 7 bits reserved.

3.5 Program State Registers

Register(s)	Width	Purpose
PC	50 bits	Program counter
SP1-SP10	50 bits each	Stack pointers for up to 10 data stacks

Register(s)	Width	Purpose
SB1-SB10	50 bits each	Stack base addresses (lower bounds)
SL1-SL10	50 bits each	Stack limit addresses (upper bounds)
SFP1-SFP10	50 bits each	Stack frame pointers, one per data stack
STK_LNK_F1-STK_LNK_F10	50 bits each	Forward link to next stack segment (0 = none)
STK_LNK_B1-STK_LNK_B10	50 bits each	Back link to previous stack segment (0 = first segment)
SSP	50 bits	Shadow stack pointer (KMI-protected, read-only from app threads)

3.6 Thread and Core Identity

Register	Width	Access	Purpose
THREAD_ID	10 bits	read-only	Thread identity (1-5)
CORE_ID	10 bits	read-only	Core identity
PRIVILEGE_LEVEL	3 bits	read-only (app)	Current privilege level (000 invalid)
SMT_MODE	10 bits	read-only	Current threading mode (ST through SMT-5)
THREAD_STATE	10 bits	read-only	Running, waiting, stalled
USER_ID	10 bits	read-only (app)	Current user session identity (1-10)
CORE_TYPE	10 bits	read-only	Core type: app (1), kernel (2), AI (3), embedded (4)

3.7 Memory Registers

Register	Width	Access	Purpose
BASE	50 bits	read-only (app)	Memory base address
BOUNDS	50 bits	read-only (app)	Memory upper limit
ADDR_MODE	1 bit	read-only	0 = direct (base + bounds), 1 = virtual
VM_TABLE_BASE	50 bits	read-only (app)	Page table root (active when ADDR_MODE=1)
VM_PAGE_SIZE	10 bits	read-only (app)	Page size configuration (active when ADDR_MODE=1)

3.8 Shared Memory Windows

Five shared memory windows per thread enable inter-process and device memory sharing without going through virtual memory:

Register set	Width	Access	Purpose
SHR_BASE_1-SHR_BASE_5	50 bits each	read-only (app)	Window base addresses
SHR_LIMIT_1-SHR_LIMIT_5	50 bits each	read-only (app)	Window upper bounds
SHR_ACCESS_1-SHR_ACCESS_5	3 bits each	read-only (app)	Access rights (R/W/X)

3.9 Message Passing

Register	Width	Access	Purpose
MSG_SEND	50 bits	read/write	Address for writing requests to the kernel core queue

Register	Width	Access	Purpose
MSG_RECV	50 bits	read/write	Address for reading kernel core responses
MSG_STATUS	10 bits	read-only (app)	Queue status: pending / complete / error

3.10 Exception Registers

Register	Width	Access	Purpose
EXCEPTION_PC	50 bits	read-only	PC at time of fault (hardware-latched)
EXCEPTION_CODE	10 bits	read-only	Fault type code (hardware-latched)

3.11 Performance Counters

Register	Width	Access	Purpose
CYCLE_COUNT	64 bits	read-only	Per-thread cycle counter
INSTR_COUNT	64 bits	read-only	Per-thread instruction counter
WALL_CLOCK	64 bits	read-only	Monotonic hardware clock
TIMER_COMPARE	64 bits	kernel only	Timer interrupt target value

3.12 Hardware Discovery

Register	Width	Access	Purpose
MACHINE_CONFIG	100 bits	read-only	Hardware capability flags (see §3.13)
MODULE_CONFIG	20 bits	read-only	ISA module capability flags (see §3.14)
AI_STATUS	10 bits	read-only (app)	AI core status: idle / busy / error

3.13 MACHINE_CONFIG Layout

A 100-bit read-only register set by Level 1 firmware at boot. Bit numbering follows the Kiss MSB-first / 1-based convention:

Bit(s)	Field	Meaning
1	HAS_AI	AI core(s) present
2	HAS_DECIMAL_AI	D.MtxMulAdd supported
3	HAS_VM	Virtual memory hardware present
4	HAS_HYPERVISOR	Level 2 hypervisor supported
5	HAS_OPTICAL	Optical interconnect present
6	HAS_TERMINAL	Terminal controller present
7-10	<i>Reserved</i>	Future boolean flags
11-15	AI_CORE_COUNT	Number of AI cores (1-31)
16-20	APP_CORE_COUNT	Number of app cores (1-31)
21-25	KERNEL_CORE_COUNT	Number of kernel cores (1-31)
26-35	AI_ARRAY_ROWS	Systolic array row dimension (0-1023)
36-45	AI_ARRAY_COLS	Systolic array column dimension (0-1023)
46-50	MAX_USERS	Maximum simultaneous users (1-31)

Bit(s)	Field	Meaning
51-55	TERMINAL_COUNT	Terminal port count
56-65	MEM_CAPACITY	Total system memory in GB (0-1023)
66-75	THREADS_PER_CORE	Threads per app core (currently 5)
76-80	<i>Reserved</i>	Future hardware features
81-90	IMPL_VENDOR	Vendor ID
91-100	IMPL_VERSION	Version/revision

3.14 MODULE_CONFIG Layout

A 20-bit read-only register set by Level 1 firmware at boot alongside MACHINE_CONFIG. Reports which ISA modules the current implementation supports. Also embedded in `.kexe` and `.k5exe` binary headers as a required-modules field for loader validation.

Bit	Field	Module
1	MOD_BASE	Base (always set)
2	MOD_DECIMAL	Decimal
3	MOD_SYSTEM	System
4	MOD_ATOMIC	Atomic
5	MOD_AI	AI/Tensor
6	MOD_USERINT	User Interrupts
7	MOD_DMA	DMA/Streaming
8	MOD_SIMD	Packed SIMD
9	MOD_TXMEM	Transactional Memory
10	MOD_CRYPTO	Crypto
11-20	<i>Reserved</i>	Future modules

3.15 Privilege Levels

Level	Code	Name	Purpose
1	001	Machine	Hardware initialization, firmware, boot
2	010	KMI / Hypervisor	Hardware abstraction, partitioning
3	011	Supervisor	OS kernel (runs on kernel cores)
4	100	<i>Reserved</i>	
5	101	User	Application code (runs on app cores)
6-7	110-111	<i>Reserved</i>	

Code `000` is invalid. Kiss keeps zero as the universal invalid sentinel rather than assigning it to a privilege level.

4. Common Instruction Structure

4.1 Standard instruction layout

Every Kiss instruction uses the same 100-bit shell.

Bit Positions	Width	Field	Purpose
1-5	5 bits	CEB	Conditional Execution Bits
6-17	12 bits	Command	ISA command
18-25	8 bits	CMB	Command Modifier Bits
26-40	15 bits	Operand 1	First operand
41-55	15 bits	Operand 2	Second operand
56-70	15 bits	Operand 3	Third operand
71-85	15 bits	Operand 4	Fourth operand
86-100	15 bits	Operand 5	Fifth operand

The command field identifies the base ISA command. The Command Modifier Bits are reused by command families to encode type, direction, immediates, conditions, or other family-specific variants.

4.2 Immediate value layout (**SetImm**)

The current **SetImm** form keeps the standard shell but repurposes the operand fields. **SetImm is a special-form command** — it is the explicit, documented exception to the destination-last (Operand 5) convention.

Bit Positions	Width	Field	Purpose
1-5	5 bits	CEB	Same as standard format
6-17	12 bits	Command	SetImm
18-25	8 bits	CMB	Immediate form control
26-40	15 bits	Operand 1	Destination register
41-50	10 bits	Operand 2 (upper)	Unused padding (must be 0)
51-55	5 bits	Operand 2 (lower)	Payload bits 1–5
56-70	15 bits	Operand 3	Payload bits 6–20
71-85	15 bits	Operand 4	Payload bits 21–35
86-100	15 bits	Operand 5	Payload bits 36–50

The 50-bit payload is right-justified in the 60-bit Operand 2–5 span. **SetVal** and **SetTxt** are assembler forms that both lower to **SetImm**.

The CMB field for **SetImm** is:

CMB Bit	Meaning
1-2	Payload data type: 00 untyped, 01 Binary, 10 Integer, 11 Decimal.
3	Immediate-form indicator. Always 1 for SetImm.

CMB Bit	Meaning
4	0 = numeric (SetVal, right-aligned); 1 = text (SetTxt, left-aligned).
5–8	Unused. Must be 0.

The ISA uses CMB 1–2 to select type-aware extension/alignment when the destination is wider than 50 bits: Binary/untyped writes raw low bits with high bits zero, Integer sign-extends per Kiss reversed-sign convention, and Decimal right-aligns encoded digits at the destination’s LSD end. Text payloads (**SetTxt**) are left-aligned regardless of CMB 1–2.

4.3 Conditional execution bits

The five-bit CEB field controls whether an instruction is unconditional or conditional, and identifies which active nesting level a conditional instruction belongs to.

- **CEB-1:** conditional-execution enable. 0 = unconditional, 1 = conditional.
- **CEB-2 through CEB-5:** the encoded condition value. For **If*** nesting, this is a one-hot bitmask identifying the active nesting level (CEB-2 = first level, CEB-3 = second, CEB-4 = third, CEB-5 = fourth).

CEB Value	Meaning
00000	Unconditional / top-level
11000	First nested If level
10100	Second nested If level
10010	Third nested If level
10001	Fourth nested If level

Execution mechanism

Each thread maintains three CEB-related state elements:

- **Main CEB** — the currently active CEB value. An instruction executes only when its CEB field matches Main CEB.
- **Shadow CEB** — saved Main CEB value, restored when the current **If** block ends.
- **Depth counter** — current **If*** nesting depth (0 = top-level).

If* instructions evaluate a condition. When **true**, they copy Main → Shadow, increment Depth, and load the next nested CEB value into Main. When **false**, the nested level is never activated and the block is skipped entirely.

EndIf matches its instruction CEB against Shadow rather than Main. When the match succeeds it restores Shadow → Main and decrements Depth, closing exactly one nesting level.

If an instruction’s CEB does not match Main, the instruction is suppressed — treated as a NOP without side effects.

Nesting depth

The current implementation supports a maximum of **4** active nesting levels. Depth is tracked explicitly rather than inferred from the CEB bit pattern, so **If** / **EndIf** matching is unambiguous.

There is no separate **Else** instruction at the ISA level — **Else** is an assembler construct that expands into **If*** / **EndIf** pairs that manipulate CEB state directly.

Interaction with Jump and Call

Jump instructions can leave conditional blocks without going through the matching **EndIfs**. They include a 3-bit “CEB clear count” in CMB bits 6–8 that tells the hardware how many active nesting levels to unwind when the jump is taken. This keeps Main / Shadow / Depth consistent regardless of how many nesting boundaries the jump crosses.

Call instructions take a different approach: they save the full CEB state (along with R1–R4, the Status Flag register **SF**, and the special-register set) to the protected shadow call stack, then start the called routine in unconditional mode (Main = **00000**, Depth = 0). R5 is **not** saved by hardware because it is the parameter-passing bank. **Retrn** restores all saved state from the shadow call stack.

4.4 Command and Command Modifier Bits

The 12-bit Command field selects the base operation. The eight-bit Command Modifier Bits (CMB) field modifies the behavior of that operation without changing its command code. A single ISA command like **Add** covers many behavioral variations through its CMB encoding.

Bit numbering

CMB bits are referenced two ways throughout the documentation. The CMB-relative numbering (CMB-1 through CMB-8) is used in family descriptions; the absolute instruction-bit numbering is used when discussing the encoded instruction word.

CMB-Relative	Absolute (Instruction)
CMB-1	bit 18
CMB-2	bit 19
CMB-3	bit 20
CMB-4	bit 21
CMB-5	bit 22
CMB-6	bit 23
CMB-7	bit 24
CMB-8	bit 25

Global conventions

Three CMB assignments are consistent across most command families:

CMB 1-2 — Data type. For typed commands, these bits select which ALU processes the instruction:

CMB 1-2	Type	ALU
00	Untyped / bit-processing / not used	—
01	Binary	Binary ALU (adder)
10	Integer	Integer ALU (subtractor)
11	Decimal	Decimal ALU (matrix lookup)

CMB-6 — Set Status Flags. For families that optionally update arithmetic or compare flags:

- 0 = do not update flags
- 1 = update flags after the operation

CMB 7-8 — Immediate data type. When CMB-3 indicates that an operand is an immediate value, CMB 7-8 encode the immediate's data type using the same encoding as CMB 1-2.

Common patterns

Most command families use one of three CMB layout patterns:

Pattern A — Typed compute (arithmetic, logic, shift / rotate, compare):

- CMB 1-2 = data type
- CMB 3-5 = family-specific modifiers (immediate indicator, direction, carry, saturation, etc.)
- CMB-6 = Set Status Flags
- CMB 7-8 = immediate data type (when applicable)

Pattern B — Data movement (Load / Stor, Read / Write, Copy / Move):

- CMB 1-2 = datatype / payload subtype where the family defines one, or unused
- CMB 3-4 = addressing mode or direction
- CMB 5-8 = family-specific or unused

Pattern C — Simple families (control-flow utilities, register utilities):

- CMB 1-8 = all zero (no modifiers)

Per-family CMB layouts are documented in each family's section in the command reference.

Unused bits

Any CMB bit not defined for a given command must be set to 0. The hardware may trap or behave unpredictably on nonzero reserved bits.

4.5 Datatype and operand notation

Kiss documentation commonly uses datatype-prefixed assembler forms such as **B.Add**, **I.Add**, and **D.Add**.

- **B** = Binary
- **I** = Integer
- **D** = Decimal

Compact operand-role notation is also used:

- **R** = register ID
- **sR** = source register (read only)
- **dR** = destination register (written)
- **sdR** = source-and-destination register (read, then written in place)
- **iR** = immediate-or-register operand (CMB-3 selects)
- **asR** = source-address register (holds memory address; memory is read — Load family)
- **adR** = destination-address register (holds memory address; memory is written — Stor family)
- **o** prefix = optional operand

See <docs/isa/kiss-isa-operand-notation.md> for the complete reference.

4.6 Shared condition suffixes

Jump, Call, and If forms share a common condition vocabulary.

Suffix	Meaning	Suffix	Meaning
Z	Zero	Pos	Positive
NZ	Not Zero	NPos	Not Positive
Blw	Below	Neg	Negative
BlwE	Below or Equal	NNeg	Not Negative
LT	Less Than	EvN	Even
LE	Less Than or Equal	Odd	Odd
EQ	Equal	Ovr	Overflow
NE	Not Equal	NOvr	Not Overflow
GE	Greater Than or Equal	Und	Underflow
GT	Greater Than	NUnd	Not Underflow
AbvE	Above or Equal	Brw	Borrow

Suffix	Meaning	Suffix	Meaning
Abv	Above	NBrw	Not Borrow
		Cry	Carry
		NCry	Not Carry

IfIRQ is specific to the If family and is not part of the shared Jump/Call suffix set.

5. Control and Conditional-Execution Commands

5.1 Overview

This family manages execution state rather than performing ordinary data transformation.

5.2 Core commands

Command	Description
NOP	No operation
Brk	Breakpoint / debugger trap
Retrn	Return from subroutine, restoring protected call-frame state
ClrCF	Clear conditional-execution state

5.3 Conditional block commands

The If family begins conditional blocks and **EndIf** closes them.

Representative forms include **IfZ**, **IfNZ**, **IfLT**, **IfGE**, **IfCry**, and **IfIRQ**. The condition is encoded in the command field rather than in a shared CMB pattern.

5.4 Notes

Most commands in this family either leave all CMB bits zero or define command-specific meanings locally. They should be documented by effect first, then by bit details where relevant.

6. Jump Commands

6.1 Overview

Jump commands transfer execution without saving a return address.

- **Jmp** = absolute jump
- **FJmp** = PC-relative forward jump

- **BJmp** = PC-relative backward jump

Conditional suffixes may be appended, for example **JmpZ**, **FJmpNE**, or **BJmpGT**.

6.2 Command Modifier Bit usage

CMB Bits	Meaning
1-2	Immediate-value datatype
3	Operand 1 is immediate when set, register when clear
4	PC-relative mode
5	Backward-relative selector when PC-relative
6-8	CEB clear count

6.3 Operand usage

- **Operand 1 (iR1)**: jump target or offset
- **Operands 2-5**: unused

6.4 Notes

CMB 6-8 encodes how many active conditional-execution levels to clear when the jump executes — see §4.3 for the Main / Shadow / Depth mechanism that consumes this count.

7. Call Commands

7.1 Overview

Call commands transfer execution to a subroutine target and save protected call-frame state so a later **Retrn** can resume correctly.

- **Call** = absolute call
- **FCall** = PC-relative forward call
- **BCall** = PC-relative backward call

Conditional suffixes may be appended in the same style as Jump forms.

7.2 Command Modifier Bit usage

CMB Bits	Meaning
1-2	Immediate-value datatype
3	Operand 1 is immediate when set, register when clear
4	PC-relative mode

CMB Bits	Meaning
5	Backward-relative selector when PC-relative
6-8	Unused, must be zero

7.3 Operand usage

- **Operand 1 (iR1)**: call target or offset
- **Operands 2-5**: unused

7.4 Notes

A Call saves R1–R4, the Status Flag register (**SF**), the CEB state (Main / Shadow / Depth), and the special-register set to the protected shadow call stack, then starts the called routine in unconditional mode. R5 is not saved (parameter-passing bank). **Retrn** restores all saved state. See §4.3 for the CEB state details.

8. Data Movement Commands

8.1 Overview

This family moves data among registers, memory, I/O ports, and stacks.

8.2 Load and Store

Load and Store move raw bytes between memory and registers.

CMB Bits	Meaning
4	Direction / alignment
5	Offset enable
6	Multiplier enable

Representative assembler forms:

- **LoadL, LoadR**
- **LoadLOfs, LoadROfs**
- **LoadLOfsMul, LoadROfsMul**
- **StorL, StorR**
- **StorLOfs, StorROfs**
- **StorLOfsMul, StorROfsMul**

Operand patterns:

- Load* iR1 asR2 [sR3] [sR4] dR5
- Stor* iR1 sR2 adR3 [sR4] [sR5]

8.3 Push and Pull

Push and Pull move variable-sized values between registers and data stacks.

CMB Bits	Meaning
3	Stack selector is immediate when set
7-8	Immediate datatype for operand 1

Operand patterns:

- Push iR1 sR2 [osR3] [osR4] [osR5]
- Pull iR1 dR2 [odR3] [odR4] [odR5]

8.4 Copy and Move

Copy duplicates a source register to one or more destinations. Move performs the same transfer and then zeroes the source after all destinations are written.

CMB Bits	Meaning
1-2	Datatype
4	Direction / alignment

Operand pattern: Copy/Move sR1 [odR2] [odR3] [odR4] dR5

8.5 Read and Write

Read and Write transfer raw bytes between registers and hardware ports.

CMB Bits	Meaning
3	Immediate byte-count selector
4	Direction / alignment

Representative forms: ReadL, ReadR, WriteL, WriteR.

8.6 Register utilities

Command	Description
ClrReg	Clear one or more registers
Swap	Exchange two registers

9. Arithmetic Commands

9.1 Overview

Arithmetic families are typed and use a shared Pattern A structure.

9.2 Pattern A CMB usage

CMB Bits	Meaning
1-2	Datatype
3	Immediate indicator
4	Saturation
5	Carry / borrow or family-specific variant
6	Set Status Flags
7-8	Family-specific, often immediate datatype

9.3 Ordinary arithmetic

Base ISA commands: **Add**, **Sub**, **Mul**, **Div**.

Representative assembler forms:

- **B.Add**, **I.Add**, **D.Add**
- **B.AddCry**, **I.SubBrw**
- **D.MulSat**, **I.DivSet**

Operand pattern: <Type>.<Op> sR1 sR2 [osR3] [osR4] dR5

9.4 Fused multiply-add and divide-with-remainder

Command	Operand pattern
MulAdd	sR1 sR2 sR3 dR5
DivRem	sR1 sR2 dR4 dR5

9.5 Apply-to families

Kiss-50 **AddTo**, **SubTo**, **MulTo**, and **DivTo** apply one input value to one in-place target register. Operand 1 may be a value register or compact immediate value; CMB-5 selects the form.

Operand pattern: <Type>.<Op>To iR1 [osdR2] [osdR3] [osdR4] sdR5

9.6 Increment and Decrement

Incr and **Decr** are amount-driven in-place update forms.

Operand pattern: <Type>.Incr/Decr iR1 [osdR2] [osdR3] [osdR4] sdR5

10. Rotate and Shift Commands

10.1 Overview

Kiss provides three related families.

ISA Command	Description
Rot	Rotate
Shift	Logical shift
SShift	Signed shift

10.2 Command Modifier Bit usage

CMB Bits	Meaning
1-2	Datatype
3	Immediate count selector
4	Direction, left or right
5	Through-carry selector
6	Set Status Flags
7-8	Immediate datatype

10.3 Assembler naming pattern

<Type>.<Family><Direction>[Set]

Examples include **B.RotL**, **I.ShiftRSet**, and **I.SShiftR**. **Rot** and **Shift** accept all three data types; **SShift** is Integer only.

10.4 Operand usage

- **Operand 1 (iR1)**: count
- **Operand 5**: required target register
- **Operands 4, 3, 2**: optional additional targets

11. Logic and Bit-Processing Commands

11.1 Overview

This section covers typed logic commands and the untyped Bit family.

11.2 Logic-family CMB usage

CMB Bits	Meaning
1-2	Datatype
6	Set Status Flags
Others	Unused, must be zero

11.3 Ordinary logic

ISA Command	Description
And	Bitwise AND
AndN	AND with inverted second operand
NAnd	NAND
Or	OR
OrN	OR with inverted second operand
NOr	NOR
XOr	XOR
NXOr	XNOR

Operand pattern: sR1 sR2 dR5

11.4 Not and Neg

- **Not** performs in-place bitwise inversion and may affect multiple registers when Set is not active.
- **Neg** performs in-place arithmetic sign inversion on a single register (no fanout).

11.5 Bit-processing overview

ISA Command	Description
BitSet	Set or clear a single bit
BitTst	Test a single bit
BitFlip	Invert selected bit(s)
BitRead	Extract a bit field
BitWrite	Insert a bit field
BitClear	Clear a bit field
BitCntTot	Count total set or clear bits
BitCnt	Count leading or trailing set or clear bits
BitFind	Find first or last set bit
BitRev	Reverse bit order
ByteRev	Reverse byte order

Bit-processing forms are untyped and generally use 00 in the datatype position.

12. Test, Compare, and Bounds Commands

12.1 Overview

Test, **Compare**, and **ChkBnds** evaluate values and set flags rather than producing a destination-register result.

12.2 Shared CMB usage

CMB Bits	Meaning
1-2	Datatype
6	Set Status Flags
Others	Unused, must be zero

12.3 Operand usage

Command	Operand usage
Test	sR1
Compare	sR1 sR2
ChkBnds	sR1 sR2 sR3

13. Conversion Commands

13.1 Overview

Cvt converts a value from one datatype to another.

13.2 Command Modifier Bit usage

CMB Bits	Meaning
1-2	Source datatype
7-8	Destination datatype
3-6	Unused, must be zero

13.3 Assembler naming pattern

<SourceType>.<DestType>.Cvt

Examples: B.I.Cvt, I.D.Cvt, D.B.Cvt.

13.4 Operand usage

- **Operand 1 (sR1)**: source register
- **Operand 5 (dR5)**: destination register

14. Special Immediate Forms and Scope Notes

14.1 SetVal and SetTxt

SetVal and **SetTxt** are readable assembler forms for the single ISA command **SetImm**. The assembler selects between them based on the literal's source form; both lower to the same ISA command with CMB-4 set according to the form chosen.

SetImm is a special-form command. The destination register is carried in Operand 1 (not Operand 5), and the 50-bit payload is right-justified in Operands 2-5 (bits 41-50 are unused padding).

CMB layout (see also §4.2):

CMB Bit	Meaning
1-2	Unused. Must be 0.
3	Immediate-form indicator. Always 1.
4	0 = numeric (SetVal, right-aligned); 1 = text (SetTxt, left-aligned).
5-8	Unused. Must be 0.

CMB bits 1-2 carry the payload data type for **SetImm**: **00** untyped, **01** Binary, **10** Integer, **11** Decimal. The assembler converts the literal into the 50-bit payload and sets CMB 1-2 from the source prefix. During execution, the CPU uses those bits for type-aware extension/alignment when the destination is wider than 50 bits: Binary/untyped writes raw low bits with high bits zero, Integer sign-extends per Kiss reversed-sign convention, and Decimal right-aligns encoded digits at the destination's LSD end. **SetTxt** text payloads are left-aligned regardless of CMB 1-2.

14.2 Scope notes

This manual covers the active Kiss ISA command set. Simulator helpers, VM conveniences, atomic operations, AI and tensor instructions, and related material outside the core ISA scope are covered in separate documents.

15. Reference Tables

15.1 Datatype codes

CMB Value	Datatype
00	Untyped / bit-processing / not used
01	Binary
10	Integer
11	Decimal

15.2 Family quick reference

Family	ISA Command(s)	Key CMB usage
Control	NOP, Brk, Retrn, ClrCF, If*, EndIf	Mostly command-defined; many all-zero
Jump	Jmp*	1-2 imm type, 3 imm, 4 relative, 5 backward, 6-8 CEB clear
Call	Call*	1-2 imm type, 3 imm, 4 relative, 5 backward
Load / Store	Load, Stor	4 direction, 5 offset, 6 multiplier
Push / Pull	Push, Pull	3 imm, 7-8 imm type
Copy / Move	Copy, Move	1-2 datatype, 4 direction
Arithmetic	Add, Sub, Mul, Div, MulAdd, DivRem, *To, Incr, Decr	Pattern A
Rotate / Shift	Rot, Shift, SShift	1-2 datatype, 3 imm, 4 direction, 5 carry, 6 set
Logic	And, Or, XOr, Not, Neg, related forms	1-2 datatype, 6 set
Bit-processing	Bit*, ByteRev	Untyped family-specific usage
Test / Compare / Bounds	Test, Compare, ChkBnds	1-2 datatype, 6 set
Conversion	Cvt	1-2 source type, 7-8 dest type
Immediate load	SetImm	special-form (dest in Opr 1, 50-bit payload in Opr 2-5), 4 val/text selector

15.3 Closing note

The Kiss ISA is deliberately regular. The manual-numbered family structure, shared 100-bit instruction shell, and repeated operand conventions are all part of that design. The goal is to keep the hardware decoder simple while keeping the command surface readable to humans.